A F M S

AIRPORT FLIGHT MANAGEMENT SYSTEM

CST2550 Group Coursework

INTRODUCTION

AFMS is a web app for managing, searching, and viewing flight information. The goal was to make it simple for anyone to use, but strong enough to handle real data. We wanted to give users a clear way to add, edit, and remove flights, and to search for flights by number or destination. The project was built from scratch, so we designed our own data structures and wrote all the code ourselves. We worked as a team, split up the work, and made sure everyone contributed to both the code and the planning.

We focused on making the app reliable and easy to use. The interface is clean, with all the main features easy to find. We also made sure the backend is solid, with our own logic for storing and searching flights. We tested everything as we went, so bugs didn't pile up. The project gave us a chance to practice real teamwork and learn how to build a full system from the ground up.

This report covers how we planned the project, how we designed the system, the algorithms and data structures we used, how we set up the database, how we tested the app, what the user interface looks like, and what we learned along the way.

DESIGN

Data Model

A flight is stored as a `Flight` object. Key fields are:

- `Id` : database identifier
- `FlightNumber` : flight code, required (e.g. BA123)
- `Airline` : airline name, required
- `Destination` : arrival airport or city, required
- `DepartureTime`, `ArrivalTime` : scheduled times; arrival must be after departure
- `Gate` : optional
- `Terminal` : terminal identifier (1–5)
- `AircraftType` : optional
- `Status` : e.g. Scheduled, Delayed; required
- `IsManualEntry` : true when a user entered or edited the record (prevents automatic overwrite)

Before saving, the model is validated to ensure required fields are present and that times are consistent.

Data structure and justification

Production storage uses EF Core and SQLite for reliable persistence and simple deployment (Microsoft, 2026b; SQLite, 2026). EF Core allows LINQ queries, easy pagination, and index use. For the coursework requirement to design a custom data structure, we implemented an array-based list to show how add, remove and search operations work at the algorithmic level.

Why both: the array-list demonstrates algorithm design and complexity analysis; the database provides a practical, robust store for the web app and supports efficient searching for the expected dataset size.

DESIGN

Key algorithms (pseudocode)

Note: pseudocode below is language-agnostic and not C# or SQL.

Add flight (database)

```
function AddFlight(flight):
 if not Validate(flight):
 return error
 database.Flights.Add(flight)
 database.SaveChanges()
 return success
```

Remove flight (database)

```
function RemoveFlight(id):
 flight = database.Flights.Find(id)
 if flight == null: return not found
 database.Flights.Remove(flight)
 database.SaveChanges()
 return success
```

Array-based list (coursework)

```
function ListAdd(flight):
  if size == capacity: resizeArray()
  array[size] = flight
  size = size + 1


function ListRemove(flightNumber):
  for i from 0 to size-1:
    if array[i].FlightNumber == flightNumber:
      shift left elements from i+1 to size-1
      size = size - 1
      break


function ListSearch(query):
  results = []
  for i from 0 to size-1:
    if query in array[i].FlightNumber or query in
array[i].Destination:
      results.append(array[i])
  return results
```

Advanced search (high level, mirros `FlightSearchService`)

```
Function AdvancedSearch(filters):
  apiFlights = fetch external API for date range
  manualFlights = load database flights where
IsManualEntry
  allFlights = mergeManual(apiFlights, manualFlights)
  filtered = applyFilters(allFlights, filters) //
airline, flight, terminal, status, dates, time range
  sorted = applySorting(filtered, filters.sortBy)
  paged = paginate(sorted, filters.page,
filters.pageSize)
  return paged
```

DESIGN

Complexity analysis

This section summarises how the main operations scale with the number of flights, in plain language.

- Database operations: inserting a flight and looking up by primary key are effectively constant time for our use (O(1) average) because the database and its indexes do most of the work.

- Array-based list (coursework prototype):
  - Add: amortised O(1) — most adds are fast; occasionally the whole array must be resized (an O(n) step).
  - Remove: O(n) — removing an item may require shifting later elements.
  - Search: O(n) — we scan items one by one for matches.

- Advanced search pipeline (merging API and manual records):
  - Merge lists: O(a + m), where a = API results and m = manual records.
  - Filter merged list: O(k), where k is the number of items after merging.
  - Sort results: O(k log k).
  - Return a single page (pagination): slicing the sorted list is O(1) work to select the page.

For the expected dataset size (hundreds to a few thousand flights) these costs are acceptable. For much larger datasets we would use indexed or full-text search to keep queries fast.

TESTING

Testing approach

We used MSTest for unit tests (Microsoft, 2026c) and manual testing for the user interface. Unit tests cover model validation, controller actions and the search service. Manual tests check form behavior, error messages and the advanced search experience.

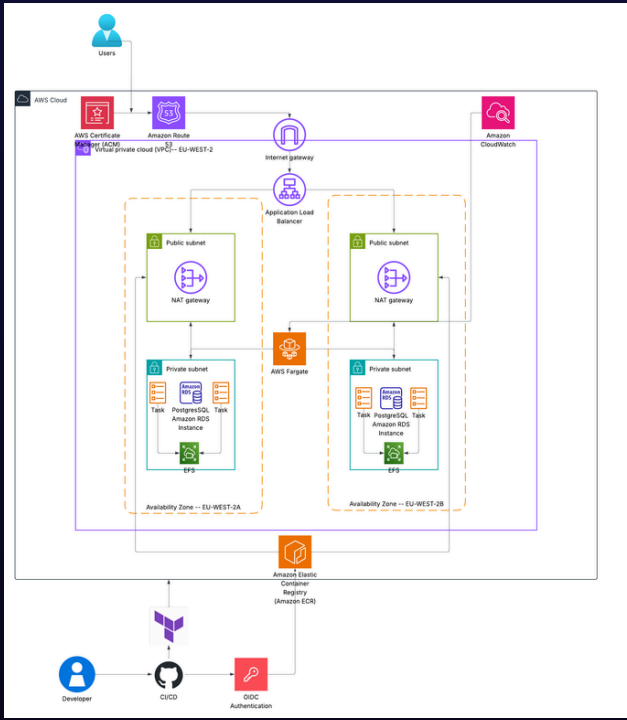
Test cases

Area	Before	After	What changed
Dashboard Merge	O(m x n)	O(m + n) *	Dictionary keyed on flight number. Legacy pass in HomeController still outstanding.
Advanced Search Merge	O(m x n)	O(m + n)	Same Dictionary-based merge used as the dashboard. No legacy pass remaining.
Advanced Search Windows	Sequential O(w)	Parallel O(1)*	Task.WhenAll perceived wait is one window. Total API work unchanged.
Background Sync	O(m x n)	O(n log m)	Composite index on FlightNumber + DepartureTime replaces full-table scan per incoming flight.
Dashboard Outstanding	O(m x n)	Not Yet Removed	Legacy nested match in HomeController.cs still needs to be replaced with the Dictionary lookup.

CLOUD ARCHITECTURE DIAGRAM

AWS Infrastructure for AFMS

The diagram above shows the cloud infrastructure we used to deploy and run the Airport Flight Management System in a real-world environment for our university project, emphasizing resilience and operational continuity. Our setup is based on AWS managed services to keep the system reliable and scalable. Users access the application through a secure load balancer in front of our app running on AWS Fargate. Resources are split across two availability zones for high availability, so if one zone fails, the other can continue serving traffic.



Each zone includes public and private subnets. Public subnets handle incoming traffic and NAT gateways, while private subnets host the main application tasks and database services. We use Amazon RDS for PostgreSQL, with managed backups enabled, and Amazon EFS for shared persistent storage between tasks when required.

The key deployment flow is image-based: the latest build image of the application is pushed to Amazon ECR, and the CI/CD pipeline deploys that latest image to the running service. Monitoring is handled through Amazon CloudWatch so we can continuously track health, performance, and operational issues.

This architecture reduces manual infrastructure management, improves security, and supports scaling as usage grows. It also gave our team practical experience in designing cloud-native systems for production-style workloads.

CONCLUSION

AFMS is a straightforward, reliable web application for creating, viewing and searching flight information. It combines external API results with manually entered records and protects user edits using the IsManualEntry flag, so manual changes are not overwritten by automatic syncs.

The team delivered a clean, easy to use interface, an AdvancedSearch pipeline that normalises input and merges API and database results, and a coursework prototype (an array based list) to demonstrate algorithmic behaviour and complexity. Server side validation and unit tests help keep core logic correct while manual UI checks ensured forms and flows are clear for users.

Current limitations include the absence of authentication and roles and permissions, and no live updates to notify clients of changes. For a production system we would add secure authentication and permission checks, switch to indexed or full text search for scalability, introduce live updates for clients, and run automated integration and end to end tests in CI with containerised deployments.

Overall, the project met its learning goals: we built a full system from scratch, practised team collaboration, and produced a solid foundation that can be extended with security, scaling, and deployment improvements.

REFERENCES

Microsoft Docs - *C# Lists*
- https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/arrays/

Entity Framework Core Documentation - *Entity Framework Core Documentation*
- https://learn.microsoft.com/en-us/ef/core/

Microsoft Docs - *MSTest Documentation*
- https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-with-mstest

GeeksforGeeks - *Data Structures - Array*
- https://www.geeksforgeeks.org/array-data-structure/

TutorialsPoint. (2026). *C# - Classes & Objects*
- https://www.tutorialspoint.com/csharp/csharp_classes.htm

Stack Overflow - *How to Implement a Custom List in C#*
- https://stackoverflow.com/questions/2629021/how-to-implement-a-custom-list-in-c

Software Testing Help - *Unit Testing in .NET*
- https://www.softwaretestinghelp.com/unit-testing-dotnet/

JetBrains - *Best Practices for Unit Testing in C#*
- https://blog.jetbrains.com/dotnet/2022/03/01/best-practices-unit-testing-csharp/

CodeProject - *Building a Simple CRUD Application in ASP.NET Core*
- https://www.codeproject.com/Articles/5270837/Building-a-Simple-CRUD-Application-in-ASP-NET-Core

MDN Web Docs - *Accessibility - MDN*
- - https://developer.mozilla.org/en-US/docs/Web/Accessibility